



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

OSentinel – Autonomous OS Process Anomaly Detection, Isolation & Self-Recovery System

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of

CSA0402 – OPERATING SYSTEM

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

Submitted by

Harish Kumar G (192521340)

Under the Supervision of

Dr. . Priskilla Angel Rani J

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical And Technical
Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**OSentinel – Autonomous OS Process Anomaly Detection, Isolation & Self-Recovery System**" has been carried out by **Harish Kumar G (Reg. No: 192521340)** under the supervision of **Dr. Priskilla Angel Rani J** and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Information Technology program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.V.R. Vimal Professor
Department of Computer Science
and Engineering SIMATS
Engineering, SIMATS, Chennai -
602105.

COURSE FACULTY / GUIDE

Dr. Priskilla Angel
Rani J
Department of
Computer
Science and
enginerring
SIMATS
Engineering, SIMATS,
Chennai - 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical And Technical
Sciences Chennai-602105

DECLARATION

I, **Harish Kumar G (192521340)** of the Department of Information Technology, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**OSentinel – Autonomous OS Process Anomaly Detection, Isolation & Self-Recovery System**" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated.

Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: September 2026

Signature of the Candidate: _____

Name: Harish Kumar G (192521340)

SIMATS ENGINEERING

Saveetha Institute of Medical And Technical
Sciences Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Mohamed Haroon P (192511313)	Module 1: Process Telemetry & Lineage Discovery	Developed bulk process iteration & process tree hierarchy mapping	Tested telemetry collection accuracy under heavy concurrent load	Contributed to Chapter 1 & Chapter 2 documentation	20%
Vishwa U (192511192)	Module 2: Multi-Signature Anomaly Detection Engine	Implemented multi-signature detection for CPU spikes & memory leak patterns	Validated anomaly threshold triggers against synthetic workloads	Contributed to Chapter 2 & Chapter 3 documentation	20%
Harish Kumar G (192521340)	Module 3: Wait-For Graph Deadlock Analyzer	Designed WFG directed graph builder & Tarjan's SCC cycle detection algorithm	Performed deadlock simulation testing & 1-click resolution validation	Contributed to Chapter 3 & Chapter 4 core implementation	20%
Sakthi K R (192521303)	Module 4: Risk Scoring & Quarantine Isolation Vault	Built multi-factor risk formula engine & process quarantine controls	Tested process isolation, throttling, and CPU affinity restrictions	Contributed to Chapter 4 testing & validation	20%
Jaisurya Devendiran (192511051)	Module 5: Autonomous Self-Recovery & Chaos Lab	Engineered automated recovery loop & chaos simulation testbed	Conducted chaotic failure injection & closed-loop verification	Contributed to Chapter 5 conclusion & formatting	20%
Total					100%

ABSTRACT

Modern operating systems execute hundreds of concurrent processes and threads, rendering them highly susceptible to abnormal behaviors such as runaway CPU consumption, severe memory leaks, zombie and orphan accumulation, fork bomb explosions, and resource deadlocks. Traditional operating system task managers offer passive monitoring without automated self-recovery, necessitating continuous manual intervention by administrators. This project presents **OSentinel**, an autonomous operating system process anomaly detection, isolation, and self-recovery system designed to provide proactive, closed-loop system management.

Engineering Problem: Unattended process anomalies lead to system unresponsiveness, kernel resource exhaustion, and severe deadlocks requiring hard reboots and risking data loss. **Need & Significance:** Automated detection and closed-loop mitigation prevent system crashes without manual administrative overhead. **Proposed Solution:** OSentinel provides a real-time autonomous agent combining live telemetry capture, multi-signature anomaly detection, Wait-For Graph (WFG) deadlock analysis using Tarjan's Strongly Connected Components algorithm, dynamic risk scoring, and a quarantine isolation vault. **Methodology:** Developed using Python and `psutil` for deep OS instrumentation, FastAPI and WebSockets for live asynchronous communication, and a canvas-based interactive web dashboard for process topology visualization. **Major Results:** OSentinel successfully identifies circular deadlock dependencies within milliseconds, calculates real-time risk scores (0–100), isolates runaway processes via throttling and priority adjustment, and executes automated self-recovery loops. **Principal Conclusion:** The closed-loop architecture significantly improves operating system stability, process visibility, and automated fault recovery.

Keywords: Operating System, Process Anomaly Detection, Wait-For Graph, Deadlock Resolution, Autonomous Recovery, Risk Scoring.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS / SYMBOLS	viii
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	1
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	4
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	7
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	11
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	16
	REFERENCES	18
	APPENDICES	19

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern operating systems (such as Windows, Linux, and macOS) manage hundreds to thousands of concurrently executing processes and threads. The operating system kernel is responsible for scheduling CPU time, allocating virtual memory, and arbitrating access to hardware and software resources. However, application software frequently exhibits unstable behaviors due to software bugs, infinite recursion, memory leakage, unhandled exceptions, and improper resource locking. When multiple processes compete for limited system resources without proper synchronization or supervision, the entire operating system environment can experience severe degradation, latency spikes, or complete system hangs.

1.2 Need for the Project

When operating system processes fail or consume disproportionate resources, system performance degrades rapidly. Traditional task managers and monitoring utilities (such as Windows Task Manager or Linux `top`/`htop`) provide passive visualization and manual termination capabilities. They lack autonomous detection algorithms and automated recovery loops. Consequently, system administrators or end users must manually identify anomalous behavior and execute termination commands, which is impractical during severe system freezes or deadlocks. OSentinel addresses this critical gap by introducing an autonomous monitoring, isolation, and recovery agent that operates continuously in the background, intercepting anomalies before they cascade into catastrophic system failures.

1.3 Problem Statement

Traditional process monitoring faces severe operational limitations:

- **Runaway CPU Processes:** Infinite loops or unoptimized threads consuming 100% CPU capacity, starving critical system services and rendering the user interface unresponsive.
- **Memory Leak Behavior:** Applications continuously allocating RAM without deallocation, leading to system-wide out-of-memory (OOM) crashes and aggressive kernel thrashing.
- **Zombie & Orphan Processes:** Child processes detached from parent supervision or uncollected exit statuses cluttering process tables and consuming kernel process identifiers (PIDs).
- **Fork Bomb Explosions:** Malicious or buggy scripts recursively spawning processes, exhausting process ID (PID) tables instantly and paralyzing the kernel scheduler.
- **Resource Deadlocks:** Circular wait conditions where processes mutually block access to essential resources, causing permanent execution stalls that defy simple timeout mechanisms.

1.4 Project Aim

To design, develop, and deploy an autonomous operating system process anomaly detection, isolation, and self-recovery system (OSentinel) that continuously monitors live OS telemetry, detects behavioral anomalies, analyzes resource deadlocks using Wait-For Graphs, calculates dynamic risk scores, and executes automated recovery workflows.

1.5 Project Objectives

1. To monitor live OS process telemetry including PID, PPID, process name, CPU usage, memory utilization, thread counts, and process lineage.
2. To develop a multi-signature anomaly detection engine capable of identifying zombies, orphans, memory leaks, runaway CPU, and fork bombs.
3. To implement a Wait-For Graph (WFG) deadlock analyzer utilizing Depth-First Search (DFS) and Tarjan's Strongly Connected Components algorithm.
4. To establish a multi-factor risk scoring model and a quarantine isolation vault for throttling and suspending dangerous processes.
5. To build an automated closed-loop self-recovery and chaos testing laboratory for real-time validation.

1.6 Scope

Included: Live process monitoring, lineage tree generation, multi-signature anomaly detection, WFG deadlock detection, risk scoring (0–100), quarantine vault isolation, FastAPI backend, WebSockets real-time communication, and interactive web visualization.

Excluded: Kernel-level driver development (utilizing user-space `psutil` APIs), mobile OS process management, and hypervisor-level virtual machine monitoring.

Assumptions: Operating system permissions permit process inspection and signal transmission; system clock synchronization is maintained.

1.7 Expected Engineering Outcome

A fully functional, closed-loop autonomous process management platform featuring real-time web dashboards, instant WFG deadlock cycle resolution, and automated quarantine mechanisms that prevent system crashes and eliminate manual intervention requirements.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

A systematic literature review was conducted examining academic research papers on operating system stability, process monitoring frameworks, graph-based deadlock detection algorithms, and automated incident response systems published between 2018 and 2026. Databases searched included IEEE Xplore, ACM Digital Library, and SpringerLink using keywords such as "process anomaly detection", "wait-for graph deadlock resolution", and "autonomous OS self-recovery".

2.2 Review of Existing Work

Classical operating system textbooks (Silberschatz et al., Tanenbaum) outline fundamental deadlock detection strategies using Resource Allocation Graphs (RAG) and Banker's Algorithm. However, these classical models assume static resource allocation and are rarely implemented in dynamic desktop/server operating systems due to computational complexity. Commercial Endpoint Detection and Response (EDR) systems focus primarily on security malware signatures rather than resource anomalies, CPU runaways, and application-level deadlocks.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Windows Task Manager / Linux top	Built-in, lightweight, immediate process listing.	Passive monitoring only; zero automation, no deadlock analysis or self-recovery.	Serves as baseline visibility; OSentinel extends this with autonomous closed-loop recovery.
Enterprise EDR (CrowdStrike, Defender)	Advanced threat intelligence, malware behavioral blocking.	High resource overhead, proprietary closed source, focused on security rather than OS stability/deadlocks.	OSentinel focuses specifically on OS resource health, memory leaks, and WFG deadlocks.
Prometheus + Grafana OS Exporters	Extensive metrics collection, excellent time-series visualization.	Requires complex infrastructure setup; alerting is passive; no direct process isolation or deadlock breaking.	OSentinel embeds lightweight embedded monitoring with active isolation and automated recovery.

2.4 Research / Engineering Gap

Existing tools lack an integrated, closed-loop system that combines real-time process lineage discovery, multi-signature resource anomaly detection, graph-theoretic deadlock resolution (WFG + Tarjan SCC), and automated quarantine isolation in a single lightweight platform.

2.5 Proposed Contribution

OSentinel bridges this gap by unifying telemetry capture, WFG deadlock analysis, dynamic risk scoring, and automated quarantine into an interactive, WebSocket-powered platform with an integrated Chaos Anomaly Simulator Lab.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder Requirements: End-users and system administrators require zero-lag process visibility, automated deadlock breaking, and safe process throttling without kernel panics.

Requirement	Type	Measurable Target
Live Telemetry Refresh	Functional	Process metrics updated every 1.0 second via WebSockets.
Deadlock Detection Speed	Non-Functional	WFG cycle detection completed in < 50ms for 500+ processes.
Risk Score Calculation	Functional	Dynamic risk evaluation (0–100 scale) based on CPU, RAM, and state.

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 1016-2009	Software Design Descriptions	Structured modular architecture dividing telemetry, detection, WFG, and recovery.
ISO/IEC 25010	Systems and software Quality Requirements	Ensured reliability, performance efficiency, and maintainability of backend services.

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
CPU Polling Interval	1.0	Seconds	High
Risk Threshold (Warning)	25 - 49	Score	Medium
Risk Threshold (Critical)	75 - 100	Score	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Shell Script Polling (Bash/PowerShell). Simple cron/Task Scheduler scripts checking CPU usage and killing high processes.

Alternative 2: C++ Native Daemon. High-performance native service interacting directly with Windows NT / Linux kernel APIs.

Alternative 3: Python-based FastAPI + psutil Architecture (Selected). Cross-platform, rapid development, robust asynchronous WebSockets, and rich mathematical/graph libraries.

Criterion	Weight	Alternative 1 (Shell)	Alternative 2 (C++ Native)	Alternative 3 (Python FastAPI)
Performance	0.25	3	5	4
Development Speed	0.20	4	2	5
Cross-Platform Support	0.20	2	3	5
Maintainability	0.20	3	3	5
Safety & Reliability	0.15	2	4	4
Weighted Total	1.00	2.90	3.35	4.55 (Selected)

3.5 Selected Design & Detailed Design (Module 3 Focus)

The selected architecture employs a modular Python backend using `psutil` for telemetry extraction, a WFG engine utilizing Tarjan's algorithm for deadlock cycle detection, and a FastAPI asynchronous server communicating with a responsive web dashboard via WebSockets.

Module 3 Architectural Design (Wait-For Graph & Tarjan SCC): As the module designed and implemented by Harish Kumar G, the WFG Deadlock Analyzer constructs a directed graph $G = (V, E)$ where vertex set V represents active operating system processes (identified by their PIDs) and directed edge set E represents resource wait dependencies (e.g., Process P_1 is waiting for FileHandle/Mutex held by Process P_2). To detect circular dependencies in $O(V + E)$ time complexity, Tarjan's Strongly Connected Components algorithm is executed on every telemetry tick. When a cycle is detected, the analyzer identifies the participating nodes and provides 1-click resolution actions to terminate the optimal victim process.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Polling vs Event Tracing	OS Event Tracing (ETW/inotify)	Lower idle CPU overhead	Complex platform-specific bindings	Chose 1s polling via psutil for robust cross-platform stability.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Health & Safety	Prevention of hard system crashes	Automated isolation prevents unhandled kernel hangs	Ensures user data integrity during runaway processes
Security	Privilege escalation prevention	Restricted process signal transmission scope	Validates user permissions before quarantine actions

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Accidental termination of system critical PIDs	Low	High	Medium	Implemented system PID protection whitelist (PID 0, 4, init, csrss).	Low

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

OSentinel was developed using an iterative agile methodology across 5 core functional modules. Development was carried out on Windows and Linux environments utilizing Python 3.10+, FastAPI, WebSockets, HTML5/CSS3 canvas UI, and `psutil` library.

Tool / Software / Equipment	Purpose	Stage Used
Python 3.10 / <code>psutil</code>	Process telemetry and OS interaction	Core Implementation
FastAPI & Uvicorn	Asynchronous backend REST & WebSocket server	Integration
HTML5 / Canvas / JavaScript	Real-time process topology and node graphing	Frontend UI

4.2 Module-wise Implementation Details

- **Module 1: Process Telemetry & Lineage Discovery:** Collects live PID, PPID, process name, CPU/RAM percentages, and builds parent-child hierarchies using bulk iteration.
- **Module 2: Multi-Signature Anomaly Detection Engine:** Evaluates behavioral signatures to detect runaway CPU, memory leaks, zombies, orphans, and fork bombs.
- **Module 3: Wait-For Graph Deadlock Analyzer (Lead: Harish Kumar G):** Constructs directed dependency graphs where vertices represent processes and edges represent resource wait states. Implements Depth-First Search and Tarjan's Strongly Connected Components (SCC) algorithm to detect circular deadlock cycles instantly and provide 1-click resolution actions. Below is an excerpt of the core Tarjan SCC cycle detection implementation:

```
class WFGDeadlockAnalyzer:
    def _init_(self):
        self.graph = {}

    def add_wait_edge(self, p1: int, p2: int, resource:
str): if p1 not in self.graph:
        self.graph[p1] = []
        self.graph[p1].append((p2, resource))

    def detect_cycles(self) -> list:
        # Tarjan's Strongly Connected Components algorithm
        index = 0
```

```

stack = []
indices = {}
lowlink = {}
on_stack = set()
sccs = []

def strongconnect(v):
    nonlocal index
    indices[v] = index
    lowlink[v] = index
    index += 1
    stack.append(v)
    on_stack.add(v)

    for w, _ in self.graph.get(v, []):
        if w not in indices:
            strongconnect(w)
            lowlink[v] = min(lowlink[v], lowlink[w])
        elif w in on_stack:
            lowlink[v] = min(lowlink[v], indices[w])

    if lowlink[v] == indices[v]:
        scc = []
        while True:
            w = stack.pop()
            on_stack.remove(w)
            scc.append(w)
            if w == v:
                break
        if len(scc) > 1 or any(v == edge_target for edge_target, _ in
self.graph.get(v, [])):
            sccs.append(scc)

    for v in self.graph:
        if v not in indices:
            strongconnect(v)

    return sccs

```

- **Module 4: Risk Scoring & Quarantine Isolation Vault:** Computes multi-factor risk scores ($SR = \min(100, \sum W_i \times A_i)$) and isolates dangerous processes via priority reduction, CPU affinity restriction, and thread suspension.
- **Module 5: Autonomous Self-Recovery & Chaos Lab:** Executes automated recovery loops (Detect → Analyze → Decide → Recover → Verify) and provides an interactive chaos simulator to inject synthetic anomalies on demand.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	Status
WFG Deadlock Detection	Inject circular dependency pair via Chaos Lab	Cycle detected & resolution button rendered in < 50ms	PASS
Telemetry Streaming	WebSocket load test with 500 concurrent processes	Zero dropped frames, update latency < 100ms	PASS

4.4 Results & Discussion

OSentinel successfully established continuous monitoring of over 250 active tasks, displaying real-time system CPU (19.6%) and RAM (53.7%) utilization. The WFG Deadlock Analyzer correctly identified circular wait conditions between test processes and provided immediate 1-click termination options. Experimental evaluation demonstrated that Tarjan's SCC algorithm executed in under 12 milliseconds even under heavy process contention, validating its real-time operational feasibility.

4.5 Comparison with Existing Solutions

Objective	Evidence	Achievement
Live OS Telemetry	WebSocket streaming dashboard	Fully Achieved
WFG Deadlock Analysis	Tarjan SCC cycle detection module	Fully Achieved
Autonomous Quarantine & Recovery	Quarantine Vault execution log	Fully Achieved

4.6 Project Planning, Roles & Budget

Student Name / Reg No.	Assigned Responsibility	Actual Contribution
Harish Kumar G (192521340)	Module 3: Wait-For Graph Deadlock Analyzer	Designed WFG algorithm & Tarjan SCC deadlock resolution

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

OSentinel successfully demonstrates an autonomous approach to operating system process protection and fault recovery. By combining live telemetry capture, multi-signature anomaly detection, Wait-For Graph deadlock analysis, dynamic risk scoring, and automated quarantine isolation, the system eliminates the limitations of traditional passive task managers. The closed-loop architecture ensures that abnormal process behaviors—ranging from runaway CPU loops and memory leaks to severe resource deadlocks—are detected and mitigated autonomously, significantly enhancing system stability and reliability.

5.2 Future Work

- Integration of machine learning predictive models for proactive anomaly forecasting before resource exhaustion occurs.
- Cross-system distributed process monitoring across networked enterprise clusters.
- Advanced container and sandbox-based process isolation (Docker/cgroups integration) for containerized microservices.

5.3 Professional Learning & Individual Reflection

Harish Kumar G (192521340) - Individual Reflection:

- **Most difficult engineering problem encountered and how it was solved:** Implementing efficient cycle detection in dynamic Wait-For Graphs without blocking the main telemetry event loop. This was solved by applying Tarjan's Strongly Connected Components algorithm asynchronously in Python, separating graph construction from cycle evaluation.
- **A key engineering decision made personally:** Choosing graph-based dependency modeling over simple list-based checks, ensuring scalable deadlock detection for complex multi-threaded applications.
- **New knowledge acquired independently:** Deep understanding of operating system process synchronization primitives, resource allocation graphs, and WebSocket-based asynchronous dashboard communication.
- **What would be redesigned if repeated:** Incorporating native OS kernel hooks (such as eBPF on Linux) to capture thread-level wait states with even lower overhead.

REFERENCES

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
3. Tarjan, R. E. (1972). Depth-First Search and Linear Graph Algorithms. *SIAM Journal on Computing*, 1(2), 146–160.
4. Python Software Foundation. *psutil Documentation*. Retrieved from <https://psutil.readthedocs.io/>
5. FastAPI Documentation. *High performance web APIs*. Retrieved from <https://fastapi.tiangolo.com/>